

Capstone Presentation Template

A guide for your capstone presentation, around 30 minutes (with feedback, up to 45). Copy this structure into your own slides or notes; the section order below matches what the instructional staff will be listening for.

1. Title

Project name, team member names, and a short framing statement: what it is, and the core idea or values behind it.

Example: (riffing on `philosoph`, the example capstone)

This is Philosoph. The name comes from the Greek root words *philo*, meaning love, and *soph*, meaning knowledge. It is a self hosted, fully local educational quiz game system for classrooms that prioritizes fine grained data analytics, data sovereignty, and respect for time and resource constraints on educators. It is created by Jon Chin and Claude Code.

2. The Problem

This is the project you will be living with for three weeks, so make it one you actually care about. Do not start from a technology you want to practice; start from something that genuinely bothers you or a question you actually want answered. The strongest capstones come from someone in the room saying “why does this not already exist” or “I keep hitting this and it is annoying.” Lay out the specific pain points in language a non-expert could follow, then connect each one directly to how your project addresses it.

Example

Tools like Kahoot are mainstays in the classroom. However, much of their functionality is locked behind a paywall. Student data is stored on their cloud. Kahoot quizzes must also be created manually. This means that an educator’s time is split between prepping and teaching, when most would want to just be teaching.

Finally, there does not seem to be many good examples of the ethical use of AI as a coding tool (rather than a coding crutch) to build a large, multifaceted program that also incorporates human critical review and oversight at every stages.

Philosoph solves these problems by being completely open source, with a PolyForm Noncommercial License, so that any educator can use it completely free of charge. The system can be run from a single laptop, without the need for paid hosting. All analytics generated from games are saved locally, never leaving the educator's laptop. It has first class support for programmatically generated questions so that the educator has to just select a topic and near infinite questions are generated automatically, freeing up the educator to do what they do best: explaining concepts, answering questions, and guiding the class.

It is also built nearly entirely through Claude Code prompts. To fully capture the role of a software engineer in the modern age of AI, all of the prompts and response have been captured in COLLABORATION.md. It emphasizes the need for vision, architecture, creativity, and critical review.

3. Approach and Key Decisions

Your stack, your architecture, and the calls you made along the way. Do not just list technologies: say why you chose each one, and name at least one place where you deliberately went against the expected or taught approach, and what you traded off to do it.

- Frontend, backend, and data layer choices
- One or two decisions that shaped the project (a tradeoff, a departure from convention, a scope cut)

Example

This project uses Ionic for the frontend, since it is a fairly comprehensive framework that covers most needs out of the box. It uses TanStack Query for server communication since it does a good job at abstraction. I chose to use the ws library for realtime data syncing since polling and manual calls were not going to be sufficient.

Philosoph also uses Express.js for its api server. Express is an industry standard so choosing it was an easy decision. To make it easy for students to connect to the

educator's locally running instance, the project also uses cors, qrcode, and ngrok.

Although the bootcamp focused on postgres for data management, Philosoph consciously does not implement a traditional database and instead stores logs and student performance metrics in flat csv files. This is because it reduces the need to install a database service, which can be daunting for educators without a computer science background. Flat csv files are less performant than even a sqlite database but are easier to understand and manage for non technical educators.

4. Demo

Show the working result, live.

5. What Was Hard

The real obstacles, not the polished version. Not every hard problem is a bug: name a design or quality problem that had no clean answer, what made it genuinely hard, and what you tried in response, even if you are still not sure it is right.

Example

The hardest part was balancing the programmatically generated questions. They had to be educationally valuable and unambiguous. In gaming terms, they had to feel fair. Fine tuning them was hard and I still don't think they are at a good enough state right now. This is exacerbated because the knowledge domain best suited for infinite programmatic generation is math, which was not an emphasis of our bootcamp. So I had questions about standard deviation, box and whisker plots, and ... nothing else.

I responded with 2 solutions: first to accept non programmatic questions. I included pre-generated questions based on our bootcamp's slides, mostly focused on definitions and distinctions. second to establish a question plugin model so that future question modules can be developed and slotted in easily. This would empower educators in other fields, like math, to build better support for programmatic questions.

6. What's Next

What comes after this. Shows you understand who your project is really for and what you still need to learn from them: which audiences you would validate with, what you would test, and how their feedback would shape your next iteration.

Example

There are 3 main audiences for Philosoph: students, educators, and curriculum developers. I need to do further testing with all 3 to make sure that I have not overlooked their priorities. Students need to find this fun (or at least fun enough for 10-15 minute sessions on a regular basis) and need to have better visibility into their own education. Educators need trustworthy software that provides valuable feedback on the skill sets that students are falling behind in. My next step would be to run pilot tests with more, real educators and students and iterate based on the feedback they provide.

7. Where You Used AI

Where and how AI tools helped (or did not). Be specific about the division of labor: what you directed (architecture, priorities, product and interface decisions) versus what the AI generated, and point to wherever you kept a full record of that collaboration.

Example

Claude Code was used to generate and revise all the code currently in the repo. I largely controlled the tech stack, data flow, and priorities of the project, consulting with Claude at certain intersections before making a final decision. I provided insight and direction on the user interface, based on my experience as a classroom educator and student. All prompts are recorded in COLLABORATION.md

8. Feedback

Open the floor for feedback from the room. This is open ended, no fixed number of questions or set amount of time.